

記事・ ユーザー管理を効率化する **React CMS フロントエンド**

フェンリル株式会社インターンシップ 2026 / フロントエンド



田中博悠

- 三田学園高等学校 1年生
- 参加コース: フロントエンド
- React SPA を 5 日間で開発
- 趣味: バンジージャンプ

画面より先に、 扱いやすい設計を作る

API 呼び出し / 共通 UI / loader / useEffect を減らす

API 呼び出しを CMS クライアントへ

```
await cms.auth.login({ email, password });  
await cms.articles.getArticles();  
await cms.users.getUser(userId);
```

責務を分けて、画面を薄くする

データ取得

TanStack Router loader

ソート

**useArticleSort /
useUserSort**

UI

**Pagination / SearchInput /
Button**

認証切れ

**401 でトークン削除、
ログインへ戻す**



useEffect は なるべく使わない

ページ表示前の取得は loader に寄せて、
副作用の置き場所を減らしました

5 日間の振り返り

よくできた

再利用できる形にできた

- libs への分離
- 共通コンポーネント化
- loader の理解

設計で迷う時間が減りました

あと一歩

手を動かす速さを上げたい

- 開発速度
- git 操作
- 事前調査

次はもっと早く形にします

よくできたこと

設計を分けて画面を薄くした

処理を役割ごとに分けて、
画面側の責務を減らしました

似た **UI** を使い回す

- Pagination
- SearchInput
- SortDropdown
- Button

短い期間でも、あとから直しやすい形を意識できました

あと一歩 1

一日、
コンフリクト解消作業に費やしました

適当な場所からブランチを切りすぎて、push
時にコンフリクトが多発しました

次はコードを書く前に、ブランチの切り方から整えます

あと一歩 2

OpenAPI generator を見落としてしまった

自力で API クライアントを書いたあとに、
生成ツールがあると知りました

次はコードを書く前に、使えるツールを確認します

レビューで、 実装の見方が変わりました

ライブラリの使い方だけでなく、
どこに責務を置くかを学びました

フロントエンド開発で AI に頼りすぎていました

これからは、自分でも書けるように練習していきます

メンターの方々、ありがとうございました!